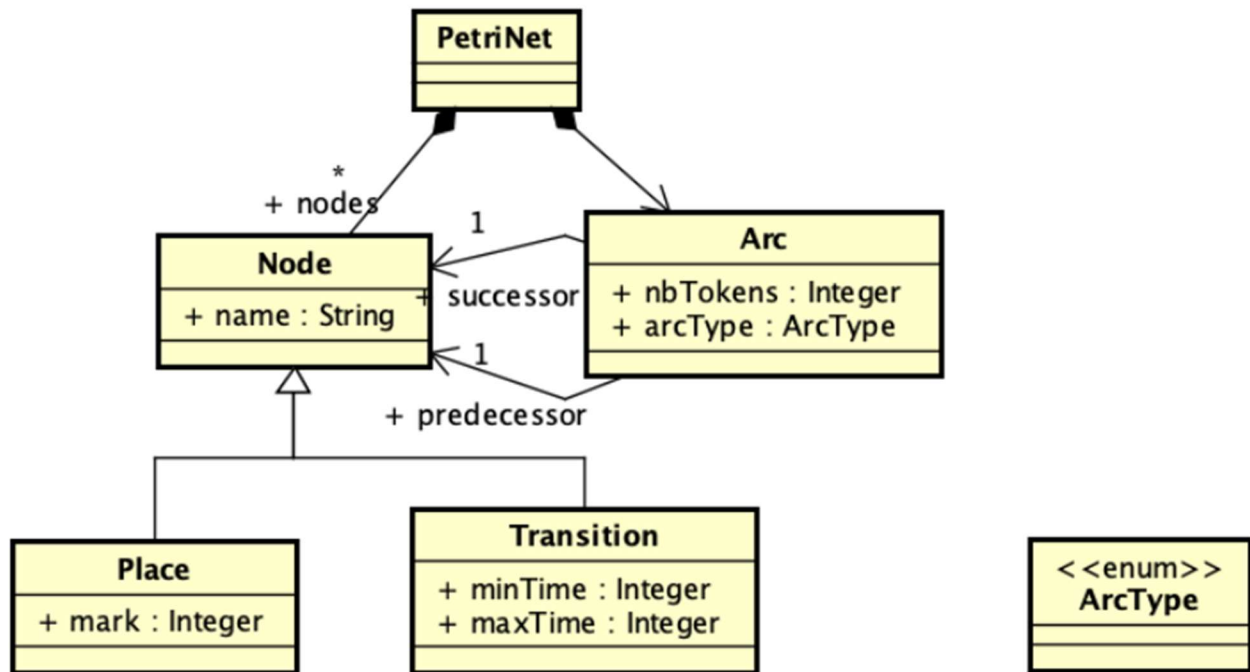


RdP



- `size()` retourne le nombre d'éléments de la collection
- `isEmpty()` retourne vrai si la collection est vide
- `notEmpty()` retourne vrai si la collection n'est pas vide
- `includes(obj)` vrai si la collection inclut l'objet obj
- `excludes(obj)` vrai si la collection n'inclut pas l'objet obj
- `including(obj)` la collection référencée doit être cette collection en incluant l'objet obj
- `excluding(obj)` idem mais en excluant obj
- `includesAll(coll)` la collection contient tous les éléments de la collection coll
- `excludesAll(coll)` la collection ne contient aucun des éléments de la collection coll

syntaxe : collection → primitive

Il n'y a pas de treizieme ` etage dans l'h `otel. ^

```
context Hotel inv : not(self.pieces->exists(p.etape=13)) context Hotel inv : self.pieces->forall(p.etape<>13))
```

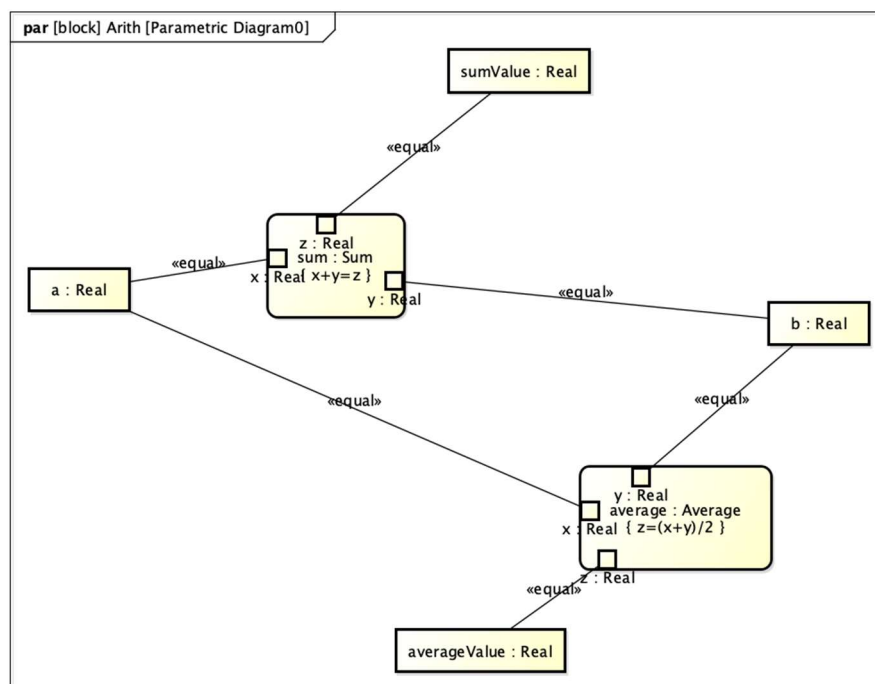
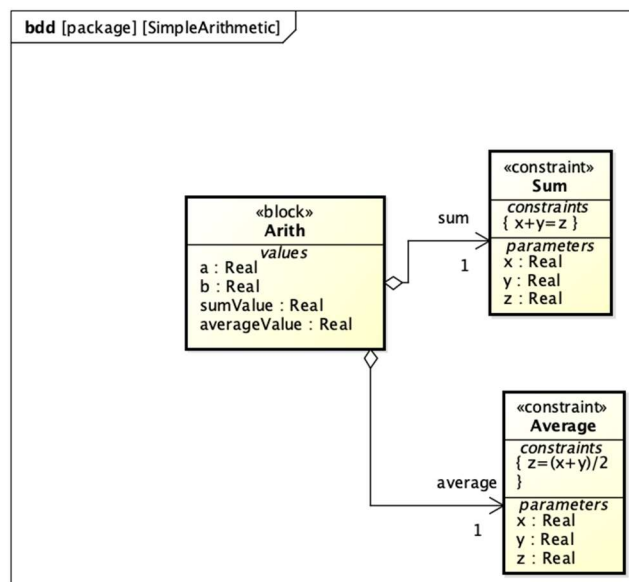
Le nombre d'occupants d'une chambre est au plus egal au nombre de lits. On tol `ere ` la

presence d'un enfants de moins de 4 ans dans un lit pliant.

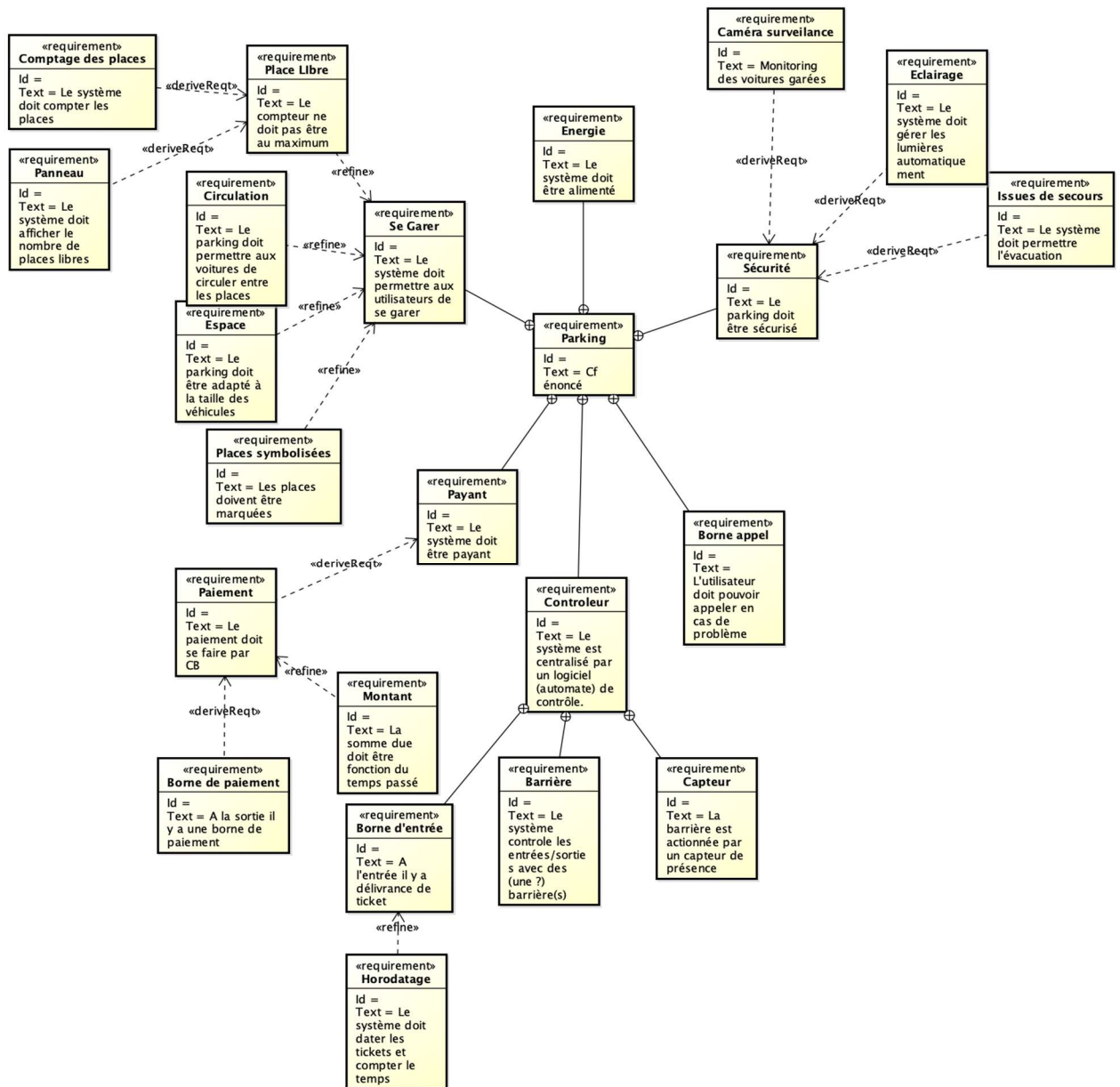
context Chambre inv : occupants->size() <= nbLits or (occupants->size() = nbLits +1 and occupants->exists(age < 4))

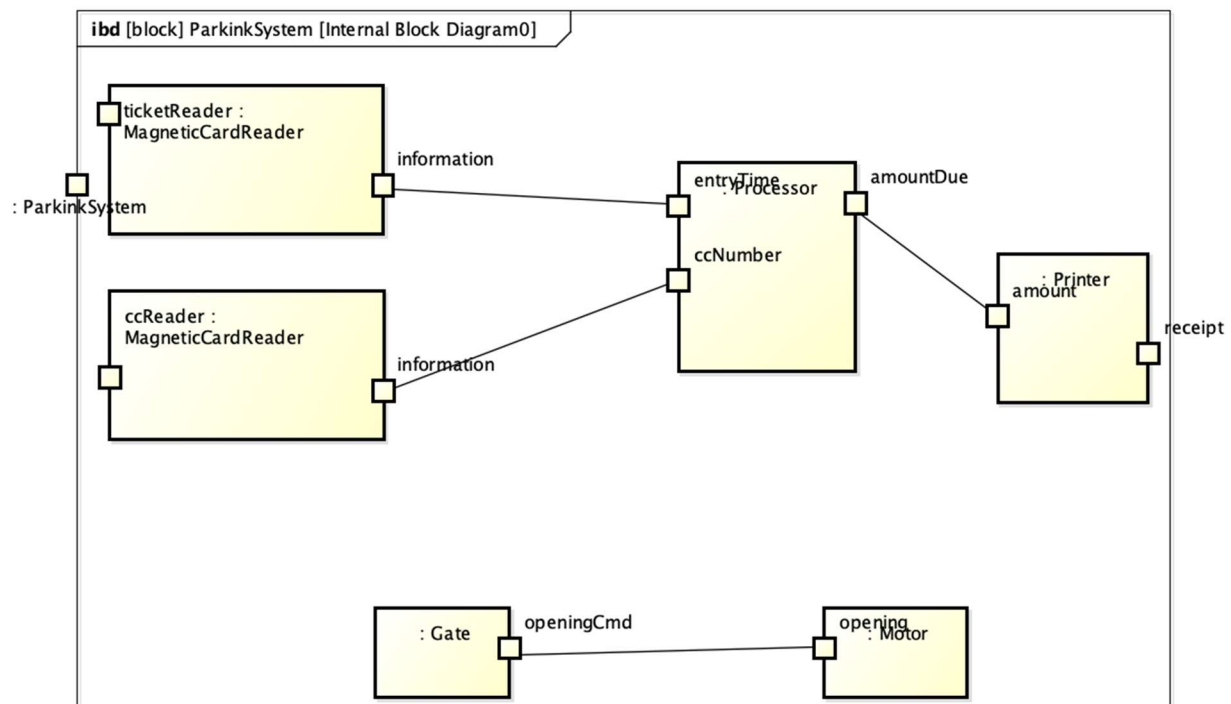
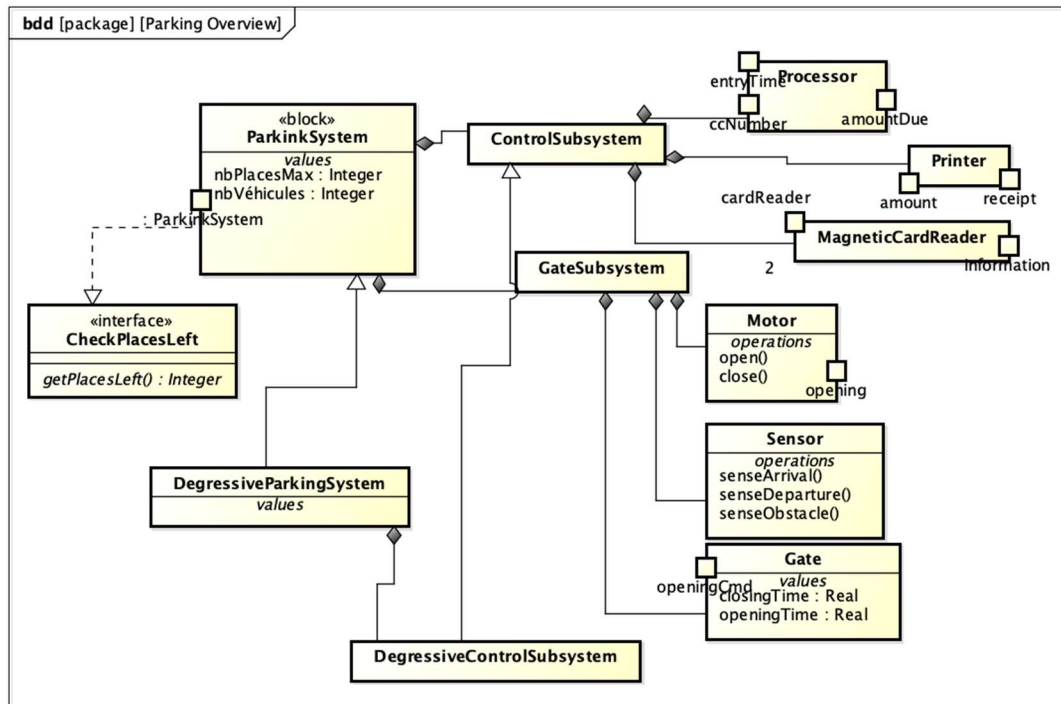
Dans chaque chambre il doit y avoir au moins une personne majeure.

context Chambre inv : self.occupants->notEmpty() implies self.occupants->exists(p | p.age >= 18)

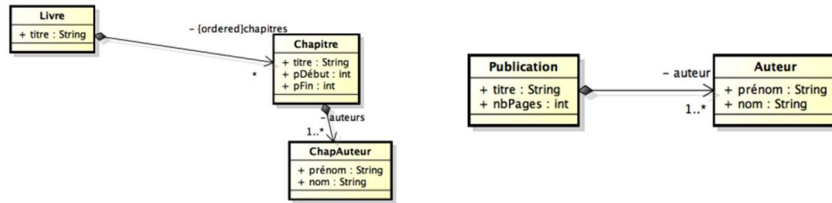


Requirement Parking





Transformations



```

module Book2Pub;
create OUT aPub : MMPublication from IN aBook : MMBook;

uses lib;

rule Chapter2Publication {
  from
    c : MMBook!Chapitre
  to
    p : MMPublication!Publication (
      titre<-c.titre;
      nbPages<-c.getNbPages(),
      authors<-authorsList,

      authorsList : distinct MMPublication!Auteur foreach(a in c.getAuthors())(
        prenom<-a.prenom,
        nom<-a.nom)
    )
}

```

```

helper context MMBook!Chapter def : getAuthors() : Set(MMBook!Author) =
  self.auteurs->collect(e | e.author)->asSet();

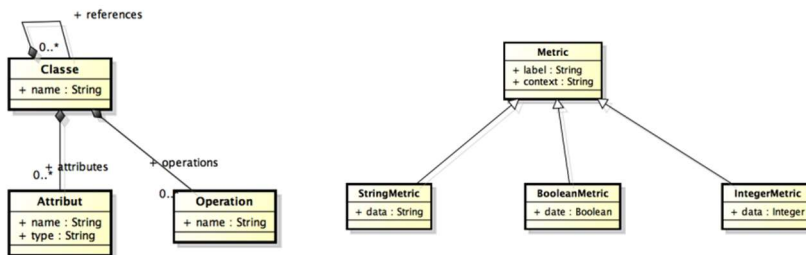
```

```

helper context MMBook!Chapter def : getNbPages() : Integer =
  self.pFin-self.pDebut;

```

2 Diagramme de classes en système de métriques

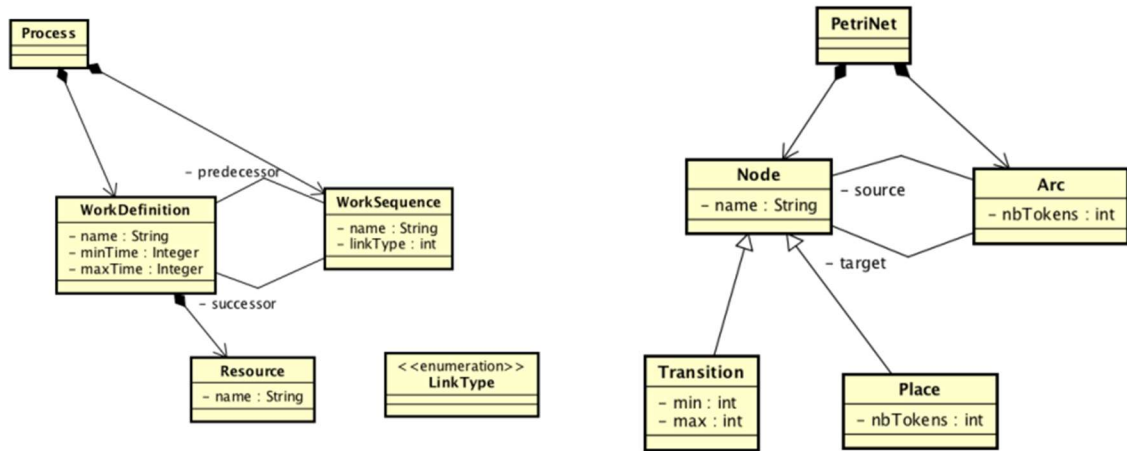


```

rule Class {
  from
    i : MMClass!Class
  to
    o1 : MMetrics!IntegerMetric (
      context <- 'Class ' + i.name,
      label <- 'Number of own attributes',
      data <- i.attributes->size()
    ),
    o2 : MMetrics!IntegerMetric (
      context <- 'Class ' + i.name,
      label <- 'Number of own references',
      data <- i.references->size()
    ),
    o3 : MMetrics!IntegerMetric (
      context <- 'Class ' + i.name,
      label <- 'Total number of methods',
      data <- i.methods->size()
    )
}

```

3 PDL en Réseau de Petri



```

module SimplePDL2PetriNet;
create OUT: PetriNet!MMPetriNet from IN: pdl!MMPdl;
rule Process2PetriNet {
from
  p: MMPdl!Process
to
  pn: PetriNet!MMPetriNet (nodes <-... , arcs <- ...)
}

rule WorkDefinition2PetriNet {
from ws: MMPdl!WorkDefinition
to
  p_notStarted : MMPetriNet!Place(name<-wd.name+'_notStarted',nbJetons<-1),
  t_start : MMPetriNet!Transition(name<-wd.name+'_start',temps_min<-0,temps_max<-(0-1)),
  ....
}

rule WorkSequence2PetriNet {
from ws:MMPdl!WorkSequence
to a_ws: MMPetriNet!Arc (...)
}
}

```

Spécifier le type de données abstrait Pile (Last In First Out) en Object-Z.

Stack[*T*]

$\uparrow(\textit{count}, \textit{INIT}, \textit{Join}, \textit{Leave})$

items : seq *T*

count : $\mathbb{N}$

INIT

items = $\langle \rangle$

Stack[*T*]

Join

$\Delta(\textit{items})$

elem? : *T*

items' = *items* $\frown$ $\langle \textit{elem?} \rangle$

Leave

$\Delta(\textit{items})$

elem! : *T*

items $\neq \langle \rangle$

items = *items'* $\frown$ $\langle \textit{elem!} \rangle$

Spécifier une horloge digitale dotée d'une alarme. Cette horloge est caractérisée par une valeur pour les heures et une valeur pour les minutes. Ces 2 valeurs représentent l'heure courante. L'alarme lorsqu'elle est activée est déterminée par un couple heures/minutes. L'horloge, au démarrage, débute à 00h00 ainsi que l'alarme mais en mode désactivée. Deux opérations de réglages sont définies : une pour régler l'heure courante et une pour régler l'alarme. Deux opérations d'affichage (respectivement heure courante et alarme). Enfin une dernière opération modifie l'heure courante pour ajouter une minute et modifier en conséquence.

DigitalClock

$\vdash (INIT)$

hours, alarmHours : [0..24]

minutes, alarmMinutes : [0..59]

alarmActivated : $\mathbb{B}$

INIT

$\neg$ *alarmActivated*

$\wedge$ *hours* = *alarmHours* = *minutes* = *alarmMinutes* = 0